CIRA Tech Platform — Test Cases & Scenarios

For QA Testers | Prism Project Management System
Base URL: `http://localhost:5062` (or as configured)
All passwords: `password123`

Test Personas

Persona	Full Name	Email	Password	Role	Contract
pm1	PM One	`pm1@ciratech.com`	`password123`	PM	Full-Time
pm2	PM Two	`pm2@ciratech.com`	`password123`	PM	Full-Time
user1	User One	`user1@ciratech.com`	`password123`	Member	Full-Time
user2	User Two	`user2@ciratech.com`	`password123`	Member	Freelancer
user3	User Three	`user3@ciratech.com`	`password123`	Member	Part-Time

Account Setup (run once before all tests):
All 5 accounts must be created before testing. Members (user1–user3) can self-register via `POST /api/auth/register`.
PM accounts require Admin to create them via `POST /api/profiles` with `"role": "PM"`.
Use the seeded admin `admin@catech.comir` / `password123` for initial setup only.

Section Index

1. Authentication
2. Profile Management
3. Projects
4. Tasks
5. Lists & Folders
6. Channels & Messages
7. Time Logs
8. Task Reports
9. Project Files
10. Invoices
11. Payroll

12. Performance Appraisals
13. Notifications
14. Dashboard
15. Role-Based Access Control (RBAC) — Negative Tests

---

1. Authentication

TC-AUTH-01 — Successful Login (pm1)

Actor: pm1
Steps:

1. `POST /api/auth/login`

```
{ "email": "pm1@ciratech.com", "password": "password123" }
```

Expected:

- Status `200 OK`
- Response contains `token` (JWT string)
- Response contains user object with `role: "PM"` and `fullName: "PM One"`
- Save token as **pm1_token** for subsequent requests

TC-AUTH-02 — Successful Login (user1)

Actor: user1
Steps:

1. `POST /api/auth/login`

```
{ "email": "user1@ciratech.com", "password": "password123" }
```

Expected:

- Status `200 OK`
- `role` field equals `"Member"`
- Save token as **user1_token**

TC-AUTH-03 — Login with Wrong Password

Actor: Any
Steps:

1. `POST /api/auth/login`

```
{ "email": "pm1@ciratech.com", "password": "wrongPassword" }
```

Expected:

- Status `401 Unauthorized`
- Error message returned

TC-AUTH-04 — Login with Non-Existent Email

Steps:

1. `POST /api/auth/login`

```
{ "email": "ghost@ciratech.com", "password": "password123" }
```

Expected:

- Status `401 Unauthorized`

TC-AUTH-05 — Register user1, user2, user3 (Members)

Actor: Each user registers themselves
Steps:

1. `POST /api/auth/register` — for user1

```
{ "fullName": "User One", "email": "user1@ciratech.com", "password": "password123" }
```

2. `POST /api/auth/register` — for user2

```
{ "fullName": "User Two", "email": "user2@ciratech.com", "password": "password123" }
```

3. `POST /api/auth/register` — for user3

```
{ "fullName": "User Three", "email": "user3@ciratech.com", "password": "password123" }
```

Expected (each):

- Status `200 OK` or `201 Created`
- Returns token + user with `role: "Member"`
- Save tokens as **user1_token**, **user2_token**, **user3_token**

> **Note:** pm1 and pm2 must be created by Admin via `POST /api/profiles` with `"role": "PM"`.

TC-AUTH-06 — Register Duplicate Email

Steps:

1. Repeat registration with `user1@ciratech.com` (already registered) **Expected:**

- Status `400 Bad Request`
- Error indicating email already exists

TC-AUTH-07 — Get Own Profile (`/me`)

Actor: user1 (using user1_token)
Steps:

1. `GET /api/auth/me`
   Header: `Authorization: Bearer {user1_token}` **Expected:**

- Status `200 OK`
- Returns user1's profile including `role`, `email`, `fullName`
- Sensitive fields like `passwordHash` must NOT appear

TC-AUTH-08 — Update Own Profile

Actor: user2
Steps:

1. `PUT /api/auth/me`

```
{ "fullName": "User Two Updated", "phone": "01001234567" }
```

Expected:

- Status `200 OK`
- Updated `fullName` reflected in response

TC-AUTH-09 — Change Password (Valid)

Actor: user1
Steps:

1. `POST /api/auth/change-password`

```
{ "currentPassword": "password123", "newPassword": "newPass456" }
```

Expected:

- Status `200 OK`

2. Verify: `POST /api/auth/login` with new password → `200 OK`

3. Reset back to `password123` after test

TC-AUTH-10 — Change Password with Wrong Current Password

Actor: user1
Steps:

1. `POST /api/auth/change-password`

```
{ "currentPassword": "incorrect", "newPassword": "anything" }
```

Expected:

- Status `400 Bad Request` or `401`

TC-AUTH-11 — Access Protected Route Without Token

Steps:

1. `GET /api/auth/me` (no Authorization header) **Expected:**

- Status `401 Unauthorized`

---

2. Profile Management

TC-PROF-01 — List All Profiles (pm1)

Actor: pm1
Steps:

1. `GET /api/profiles`
   Header: `Authorization: Bearer {pm1_token}` **Expected:**

- Status `200 OK`
- Returns array of profiles
- All registered users visible (pm1, pm2, user1, user2, user3)

TC-PROF-02 — Get Single Profile (user1)

Actor: user1
Steps:

1. First get any profile ID from TC-PROF-01
2. `GET /api/profiles/{id}` **Expected:**

- Status `200 OK`
- Profile object with `fullName`, `email`, `role`, `contractType`

TC-PROF-03 — Create New Profile (pm1)

Actor: pm1

Steps:

1. `POST /api/profiles`

```
{
  "fullName": "New Employee",
  "email": "new.emp@ciratech.com",
  "role": "Member",
  "contractType": "PartTime",
  "hourlyRate": 30,
  "password": "password123"
}
```

Expected:

- Status `201 Created`
- Profile created with provided details
- Save returned profile `id` as **newProfile_id**

TC-PROF-04 — Update Profile (pm1)

Actor: pm1

Steps:

1. `PUT /api/profiles/{newProfile_id}`

```
{ "hourlyRate": 35, "hoursPerWeek": 20 }
```

Expected:

- Status `200 OK`
- Updated values reflected

TC-PROF-05 — Member Cannot Create Profile (user1)

Actor: user1

Steps:

1. `POST /api/profiles` (same body as TC-PROF-03)

   Header: `Authorization: Bearer {user1_token}` **Expected:**

- Status `403 Forbidden`

TC-PROF-06 — Delete Profile (pm2 / Admin only)

Actor: pm2

Steps:

1. `DELETE /api/profiles/{newProfile_id}`

   Header: `Authorization: Bearer {pm2_token}` **Expected:**

- Status `204 No Content`
- Profile no longer returned in list (soft-deleted)

---

3. Projects

TC-PROJ-01 — Create Project (pm1)

Actor: pm1

Steps:

1. `POST /api/projects`

```
{
  "name": "Alpha Project",
  "description": "First test project",
  "totalBudget": 50000,
  "status": "Active"
}
```

Expected:

- Status `201 Created` or `200 OK`
- Project created, saves returned `id` as **proj1_id**
- A primary channel is **auto-created** for the project

TC-PROJ-02 — Create Second Project (pm2)

Actor: pm2

Steps:

1. `POST /api/projects`

```
{
  "name": "Beta Project",
  "description": "Second test project",
  "totalBudget": 30000,
  "status": "Active"
}
```

Expected:

- Status `201 Created`

- Save returned `id` as **proj2_id**

TC-PROJ-03 — List Projects (pm1 sees all)

Actor: pm1
Steps:

1. `GET /api/projects` **Expected:**

- Status `200 OK`
- Both proj1 and proj2 appear (Admin/PM see all)

TC-PROJ-04 — List Projects (user1 sees only assigned)

Actor: user1
Steps:

1. `GET /api/projects` **Expected:**

- Status `200 OK`
- Only projects where user1 is a member/assignee are returned
- If user1 has no assignment → empty array `[]`

TC-PROJ-05 — Add Members to Project (pm1)

Actor: pm1
Steps:

1. Get user IDs for user1, user2, user3 from profiles list
2. `POST /api/projects/{proj1_id}/members`

```
{ "user_ids": ["{user1_id}", "{user2_id}"] }
```

Expected:

- Status `200 OK`
- user1 and user2 are now project members

TC-PROJ-06 — Verify Member Sees Project After Assignment

Actor: user1
Steps:

1. `GET /api/projects` (after TC-PROJ-05) **Expected:**

- proj1 now appears in user1's project list

TC-PROJ-07 — Get Project Members

Actor: pm1
Steps:

1. `GET /api/projects/{proj1_id}/members` **Expected:**

- Status `200 OK`
- Returns array including user1, user2, and pm1 (as manager)

TC-PROJ-08 — Add user3 then Remove From Project

Actor: pm1
Steps:

1. `POST /api/projects/{proj1_id}/members` → add user3
2. Verify user3 in members list
3. `DELETE /api/projects/{proj1_id}/members/{user3_id}` **Expected:**

- After step 3: Status `200 OK` or `204`
- user3 no longer in members list

TC-PROJ-09 — Update Project (pm1)

Actor: pm1
Steps:

1. `PUT /api/projects/{proj1_id}`

```
{ "name": "Alpha Project Updated", "totalBudget": 60000, "status": "Active" }
```

Expected:

- Status `200 OK`
- Updated values visible on GET

TC-PROJ-10 — Member Cannot Create Project (user1)

Actor: user1
Steps:

1. `POST /api/projects` (same body as TC-PROJ-01)
   Header: `Authorization: Bearer {user1_token}` **Expected:**

- Status `403 Forbidden`

TC-PROJ-11 — Delete Project (pm2)

Actor: pm2
Steps:

1. Create a throwaway project first
2. `DELETE /api/projects/{throwaway_id}` **Expected:**

- Status `200 OK` or `204`
- Project not returned in list (soft-deleted)

---

4. Tasks

> **Prerequisites:** proj1 exists; user1 & user2 are members of proj1.
> A list must exist inside a folder inside proj1 — see Section 5 for setup.

TC-TASK-01 — Create Task (pm1, assign to user1)

Actor: pm1
Steps:

1. `POST /api/tasks`

```
{
  "title": "Design Login Page",
  "description": "Create mockup for login",
  "listId": "{list1_id}",
  "priority": "High",
  "estimatedHours": 8,
  "dueDate": "2026-04-30",
  "assigneeId": "{user1_id}"
}
```

Expected:

- Status `201 Created`
- `status` defaults to `"ToDo"`
- user1 receives a `TaskAssigned` notification
- Save returned `id` as **task1_id**

TC-TASK-02 — Create Task with Multiple Assignees (pm1)

Actor: pm1
Steps:

1. `POST /api/tasks`

```
{
  "title": "Build API Integration",
  "listId": "{list1_id}",
  "priority": "Medium",
  "assigneeIds": ["{user1_id}", "{user2_id}"]
}
```

Expected:

- Status `201 Created`
- Both user1 and user2 receive `TaskAssigned` notifications
- Save id as **task2_id**

TC-TASK-03 — List All Tasks (pm1)

Actor: pm1
Steps:

1. `GET /api/tasks` **Expected:**

- Status `200 OK`
- task1 and task2 in results

TC-TASK-04 — List Tasks by List (user1)

Actor: user1
Steps:

1. `GET /api/tasks/list/{list1_id}` **Expected:**

- Status `200 OK`
- Only tasks that belong to list1 and user1 is assigned to

TC-TASK-05 — Member Cannot Create Task (user1)

Actor: user1
Steps:

1. `POST /api/tasks` (same body as TC-TASK-01)
   Header: `Authorization: Bearer {user1_token}` **Expected:**

- Status `403 Forbidden`

TC-TASK-06 — Update Task Status: ToDo → InProgress (user1 as assignee)

Actor: user1
Steps:

1. `PATCH /api/tasks/{task1_id}/status`

```
{ "status": "InProgress" }
```

Expected:

- Status `200 OK`
- Task status is now `"InProgress"`

TC-TASK-07 — Submit Task for Review

Actor: user1
Steps:

1. `POST /api/tasks/{task1_id}/submit-review` **Expected:**

- Status `200 OK`
- Task status changes to `"InReview"`

TC-TASK-08 — Approve Task (pm1 moves to Done)

Actor: pm1
Steps:

1. `PATCH /api/tasks/{task1_id}/status`

```
{ "status": "Done" }
```

Expected:

- Status `200 OK`
- Task status is `"Done"`
- user1 receives a `TaskCompleted` notification

TC-TASK-09 — Non-Assignee Cannot Change Task Status (user3)

Actor: user3 (not assigned to task1)
Steps:

1. `PATCH /api/tasks/{task1_id}/status`

```
{ "status": "InProgress" }
```

Header: `Authorization: Bearer {user3_token}` **Expected:**

- Status `403 Forbidden`

TC-TASK-10 — Update Task Details (pm1)

Actor: pm1
Steps:

1. `PUT /api/tasks/{task2_id}`

```
{
  "title": "Build API Integration - Revised",
  "priority": "High",
  "estimatedHours": 12,
  "assigneeIds": ["{user2_id}"]
}
```

Expected:

- Status `200 OK`
- user1 is removed from assignees; user2 remains
- Notifications sent for reassignment changes

TC-TASK-11 — Delete Task (pm1)

Actor: pm1
Steps:

1. Create a throwaway task
2. `DELETE /api/tasks/{throwaway_task_id}` **Expected:**

- Status `200 OK` or `204`
- Task no longer in list

TC-TASK-12 — Invalid Status Transition (skip step)

Actor: pm1
Steps:

1. With task in `"ToDo"` status, try:

```
{ "status": "Done" }
```

Expected:

- Behaviour depends on implementation: either `400 Bad Request` or allowed (document actual result)

---

5. Lists & Folders

> Lists and Folders endpoints have **no auth enforced** (testing mode).

TC-LIST-01 — Create Folder for proj1

Steps:

1. `POST /api/folders`

```
{ "spaceId": "{proj1_id}", "name": "Sprint 1" }
```

Expected:

- Status `201 Created`
- Save returned `id` as **folder1_id**

TC-LIST-02 — List All Folders

Steps:

1. `GET /api/folders` **Expected:**

- Status `200 OK`
- folder1 present in results

TC-LIST-03 — Create List Inside Folder

Steps:

1. `POST /api/lists`

```
{ "folderId": "{folder1_id}", "name": "Backlog" }
```

Expected:

- Status `201 Created`
- Save returned `id` as **list1_id**

TC-LIST-04 — Create Second List

Steps:

1. `POST /api/lists`

```
{ "folderId": "{folder1_id}", "name": "In Progress" }
```

Expected:

- Status `201 Created`
- Save returned `id` as **list2_id**

TC-LIST-05 — List All Lists

Steps:

1. `GET /api/lists` **Expected:**

- Status `200 OK`
- list1 and list2 in results

---

6. Channels & Messages

TC-CHAN-01 — List Channels (pm1)

Actor: pm1
Steps:

1. `GET /api/channels` **Expected:**

- Status `200 OK`
- Global channels visible to all
- Project channels from proj1 (auto-created in TC-PROJ-01) visible to pm1

TC-CHAN-02 — List Channels (user1, assigned to proj1)

Actor: user1
Steps:

1. `GET /api/channels` **Expected:**

- Status `200 OK`
- proj1's channel visible since user1 is member

TC-CHAN-03 — List Channels (user3, not in any project)

Actor: user3
Steps:

1. `GET /api/channels` **Expected:**

- Status `200 OK`
- Only global channels (if any); proj1 channel NOT visible

TC-CHAN-04 — Create a New Channel (pm1)

Actor: pm1
Steps:

1. `POST /api/channels`

```
{ "name": "General Announcements", "isPrivate": false, "spaceId": "{proj1_id}" }
```

Expected:

- Status `201 Created`
- Save returned `id` as **chan1_id**

TC-CHAN-05 — Create Private Channel

Actor: pm1

Steps:

1. `POST /api/channels`

```
{ "name": "PM Only", "isPrivate": true, "spaceId": "{proj1_id}" }
```

Expected:

- Status `201 Created`
- Save as **chan_private_id**

TC-MSG-01 — Send Message (user1)

Actor: user1

Steps:

1. `POST /api/messages`

```
{ "channelId": "{chan1_id}", "content": "Hello team!" }
```

Expected:

- Status `201 Created`
- Message stored with user1 as sender

TC-MSG-02 — Get Messages in Channel (user2)

Actor: user2

Steps:

1. `GET /api/messages?channelId={chan1_id}` **Expected:**

- Status `200 OK`
- Returns last 50 messages including user1's message

TC-MSG-03 — Send Message with HTML Injection (user1)

Actor: user1

Steps:

1. `POST /api/messages`

```
{ "channelId": "{chan1_id}", "content": "<script>alert('XSS')</script>Hello" }
```

Expected:

- Status `201 Created`
- Stored content has `<script>` tags **sanitized/stripped**
- No raw script tags in response

TC-MSG-04 — User Cannot Read Messages from Inaccessible Channel (user3)

Actor: user3 (not a proj1 member)

Steps:

1. `GET /api/messages?channelId={chan1_id}` Header: `Authorization: Bearer {user3_token}` **Expected:**

- Status `403 Forbidden` or empty result depending on access policy

---

7. Time Logs

TC-TIME-01 — Create Time Log (user1 on task1)

Actor: user1

Steps:

1. `POST /api/timelogs`

```
{
  "taskId": "{task1_id}",
  "startTime": "2026-04-03T09:00:00",
  "endTime": "2026-04-03T13:00:00",
  "durationHours": 4,
  "isBillable": true,
  "isManualEntry": true
}
```

Expected:

- Status `201 Created`
- Save returned `id` as **log1_id**

TC-TIME-02 — Create Time Log (user2)

Actor: user2

Steps:

1. `POST /api/timelogs`

```
{
  "taskId": "{task2_id}",
  "startTime": "2026-04-03T10:00:00",
  "endTime": "2026-04-03T14:00:00",
  "durationHours": 4,
  "isBillable": false,
  "isManualEntry": true
}
```

Expected:

- Status `201 Created`
- Save as **log2_id**

TC-TIME-03 — List Time Logs (pm1 sees all)

Actor: pm1

Steps:

1. `GET /api/timelogs` **Expected:**

- Status `200 OK`
- Both log1 and log2 visible

TC-TIME-04 — List Time Logs (user1 sees own only)

Actor: user1

Steps:

1. `GET /api/timelogs` **Expected:**

- Status `200 OK`
- Only log1 (user1's own log) visible; log2 NOT included

TC-TIME-05 — Update Time Log (user1 updates own log)

Actor: user1

Steps:

1. `PUT /api/timelogs/{log1_id}`

```
{ "durationHours": 5, "isBillable": true }
```

Expected:

- Status `200 OK`
- `durationHours` updated to 5

TC-TIME-06 — User Cannot Update Another User's Log (user1 updates log2)

Actor: user1
Steps:

1. `PUT /api/timelogs/{log2_id}` (log2 belongs to user2) Header: `Authorization: Bearer {user1_token}` **Expected:**

- Status `403 Forbidden` or `404 Not Found`

TC-TIME-07 — Delete Time Log (user1 deletes own log)

Actor: user1
Steps:

1. `DELETE /api/timelogs/{log1_id}` **Expected:**

- Status `200 OK` or `204`
- log1 no longer appears in user1's logs

TC-TIME-08 — PM Can Update Any Time Log (pm1 updates log2)

Actor: pm1
Steps:

1. `PUT /api/timelogs/{log2_id}`

```
{ "durationHours": 6 }
```

Expected:

- Status `200 OK`
- Duration updated successfully

---

8. Task Reports

TC-REP-01 — Member Submits Report (user1)

Actor: user1
Steps:

1. `POST /api/reports`

```
{
  "taskId": "{task1_id}",
  "content": "Completed design mockup. Waiting for review.",
  "reportType": "submit"
}
```

Expected:

- Status `201 Created`
- Save returned `id` as **report1_id**

TC-REP-02 — Member Cannot Submit Approve/Reject Report (user1)

Actor: user1

Steps:

1. `POST /api/reports`

```
{
  "taskId": "{task1_id}",
  "content": "Approving this task.",
  "reportType": "approve"
}
```

Header: `Authorization: Bearer {user1_token}` **Expected:**

- Status `403 Forbidden`

TC-REP-03 — PM Approves Report (pm1)

Actor: pm1

Steps:

1. `POST /api/reports`

```
{
  "taskId": "{task1_id}",
  "content": "Design approved. Looks great.",
  "reportType": "approve"
}
```

Expected:

- Status `201 Created`

TC-REP-04 — PM Creates Project-Level Report

Actor: pm1

Steps:

1. `POST /api/reports`

```
{
  "spaceId": "{proj1_id}",
  "content": "Sprint 1 summary: 2 tasks completed, 1 in review.",
  "reportType": "project"
}
```

Expected:

- Status `201 Created`

TC-REP-05 — Get Reports for Task (pm1)

Actor: pm1
Steps:

1. `GET /api/reports/task/{task1_id}` **Expected:**

- Status `200 OK`
- Returns report1 and the approval report from TC-REP-03

TC-REP-06 — Get Reports for Project (pm1)

Actor: pm1
Steps:

1. `GET /api/reports/project/{proj1_id}` **Expected:**

- Status `200 OK`
- Returns project-level report from TC-REP-04

TC-REP-07 — Delete Report (author only — user1 deletes own)

Actor: user1
Steps:

1. `DELETE /api/reports/{report1_id}`
   Header: `Authorization: Bearer {user1_token}` **Expected:**

- Status `200 OK` or `204`

TC-REP-08 — Non-Author Cannot Delete Report (user2 deletes pm1's report)

Actor: user2
Steps:

1. `DELETE /api/reports/{pm1_report_id}`
   Header: `Authorization: Bearer {user2_token}` **Expected:**

- Status `403 Forbidden`

---

9. Project Files

TC-FILE-01 — Upload File to Project (user1)

Actor: user1
Steps:

1. `POST /api/projects/{proj1_id}/files`
   (multipart/form-data, attach any file ≤ 50MB)
   Header: `Authorization: Bearer {user1_token}` **Expected:**

- Status `200 OK` or `201 Created`
- Returns file metadata including download URL
- Save `fileId` as **file1_id**

TC-FILE-02 — List Project Files (pm1)

Actor: pm1
Steps:

1. `GET /api/projects/{proj1_id}/files` **Expected:**

- Status `200 OK`
- file1 in results with name and download URL

TC-FILE-03 — Download File (user2)

Actor: user2
Steps:

1. `GET /api/projects/{proj1_id}/files/{file1_id}/download` **Expected:**

- Status `200 OK`
- File bytes returned with correct `Content-Type`

TC-FILE-04 — Preview File Inline

Actor: user1
Steps:

1. `GET /api/projects/{proj1_id}/files/{file1_id}/download?inline=true` **Expected:**

- Status `200 OK`
- `Content-Disposition: inline` header

TC-FILE-05 — Upload File Exceeding 50MB

Actor: user1
Steps:

1. Attempt to upload a file > 50MB **Expected:**

- Status `400 Bad Request` or `413 Payload Too Large`

TC-FILE-06 — Delete File (pm1)

Actor: pm1
Steps:

1. `DELETE /api/projects/{proj1_id}/files/{file1_id}` **Expected:**

- Status `200 OK` or `204`
- File no longer in list

---

10. Invoices

> All invoice operations require **PM or Admin** role.

TC-INV-01 — Create Services Invoice (pm1)

Actor: pm1
Steps:

1. `POST /api/invoices`

```
{
  "invoiceNumber": "INV-2026-001",
  "userId": "{user1_id}",
  "spaceId": "{proj1_id}",
  "invoiceType": "Services",
  "taxRate": 0.14,
  "lineItems": [
    { "description": "UI Design - Sprint 1", "quantity": 8, "unitPrice": 45 }
  ]
}
```

Expected:

- Status `201 Created`
- `status` defaults to `"Draft"`
- `subTotal = 360`, `taxAmount = 50.4`, `totalAmount = 410.4`
- Save returned `id` as **inv1_id**

TC-INV-02 — Create Payroll Invoice (pm1)

Actor: pm1

Steps:

1. `POST /api/invoices`

```
{
  "invoiceNumber": "INV-2026-002",
  "userId": "{user2_id}",
  "spaceId": "{proj1_id}",
  "invoiceType": "Payroll",
  "taxRate": 0,
  "lineItems": [
    { "description": "March Payroll", "quantity": 1, "unitPrice": 2400 }
  ]
}
```

Expected:

- Status `201 Created`
- Save as **inv2_id**

TC-INV-03 — List Invoices (pm1)

Actor: pm1

Steps:

1. `GET /api/invoices` **Expected:**

- Status `200 OK`
- Both inv1 and inv2 in results

TC-INV-04 — Filter Invoices by Type

Actor: pm1

Steps:

1. `GET /api/invoices?type=Services` **Expected:**

- Only Services-type invoices returned

TC-INV-05 — Filter Invoices by Status

Actor: pm1

Steps:

1. `GET /api/invoices?status=Draft` **Expected:**

- Only Draft invoices returned

TC-INV-06 — Get Invoice Details

Actor: pm1
Steps:

1. `GET /api/invoices/{inv1_id}` **Expected:**

- Status `200 OK`
- Includes `lineItems` array with at least 1 item

TC-INV-07 — Update Invoice Status: Draft → Sent

Actor: pm1
Steps:

1. `PATCH /api/invoices/{inv1_id}/status`

```
{ "status": "Sent" }
```

Expected:

- Status `200 OK`
- Invoice status is now `"Sent"`

TC-INV-08 — Update Invoice Status: Sent → Paid

Actor: pm1
Steps:

1. `PATCH /api/invoices/{inv1_id}/status`

```
{ "status": "Paid" }
```

Expected:

- Status `200 OK`
- If inv1 is linked to a Payroll record, that payroll's status also becomes `"Paid"`

TC-INV-09 — Member Cannot Create Invoice (user1)

Actor: user1
Steps:

1. `POST /api/invoices` (same body as TC-INV-01)
   Header: `Authorization: Bearer {user1_token}` **Expected:**

- Status `403 Forbidden`

TC-INV-10 — Delete Invoice (pm1)

Actor: pm1
Steps:

1. Create a throwaway invoice
2. `DELETE /api/invoices/{throwaway_inv_id}` **Expected:**

- Status `200 OK` or `204`

---

11. Payroll

> Payroll actions require **Admin, PM, or HR** role.

TC-PAY-01 — Preview Payroll for user1

Actor: pm1
Steps:

1. Re-create log1 if deleted (TC-TIME-01)
2. `GET /api/payrolls/preview?userId={user1_id}&periodStart=2026-04-01&periodEnd=2026-04-30` **Expected:**

- Status `200 OK`
- Returns calculated breakdown: total hours, projected base pay, overtime (if FT), estimated net

---

TC-PAY-02 — Create Payroll Record for user1 (FT)

Actor: pm1
Steps:

1. `POST /api/payrolls`

```
{
  "userId": "{user1_id}",
  "periodStart": "2026-04-01",
  "periodEnd": "2026-04-30",
  "overtimeHours": 2,
  "deductions": 0
}
```

Expected:

- Status `201 Created`
- Net amount auto-calculated: `BaseSalary + (OvertimeHours × HourlyRate × 1.5) + Bonus`
- Bonus applied based on latest performance appraisal (0% if none)
- Save returned `id` as **pay1_id**

TC-PAY-03 — Create Payroll for user2 (Freelancer)

Actor: pm1

Steps:

1. `POST /api/payrolls`

```
{
  "userId": "{user2_id}",
  "periodStart": "2026-04-01",
  "periodEnd": "2026-04-30"
}
```

Expected:

- Status `201 Created`
- Net = `TotalHours × HourlyRate + Bonus`

TC-PAY-04 — Get Payroll Summary

Actor: pm1

Steps:

1. `GET /api/payrolls/summary` **Expected:**

- Status `200 OK`
- Returns counts by contract type (FT, PT, FL) and status (Draft, Approved, Paid)

TC-PAY-05 — List Payrolls

Actor: pm1

Steps:

1. `GET /api/payrolls` **Expected:**

- Status `200 OK`
- Both pay1 and pay2 present

TC-PAY-06 — Get Single Payroll with Performance Score

Actor: pm1

Steps:

1. `GET /api/payrolls/{pay1_id}` **Expected:**

- Status `200 OK`
- Includes `performanceScore` from latest appraisal (null if none)

TC-PAY-07 — Approve Payroll (pm1)

Actor: pm1

Steps:

1. `PATCH /api/payrolls/{pay1_id}/status`

```
{ "status": "Approved" }
```

Expected:

- Status `200 OK`
- Payroll status = `"Approved"`
- A **Payroll Invoice** is **auto-created** linked to this payroll

---

TC-PAY-08 — Verify Auto-Created Invoice

Actor: pm1

Steps:

1. `GET /api/invoices?type=Payroll` **Expected:**

- A Payroll invoice for user1's period appears in results

TC-PAY-09 — Mark Payroll as Paid (via invoice sync)

Actor: pm1

Steps:

1. Get auto-created payroll invoice from TC-PAY-08
2. `PATCH /api/invoices/{payroll_inv_id}/status` → `{ "status": "Paid" }` **Expected:**

- pay1 status also becomes `"Paid"` (synced)

TC-PAY-10 — Bonus Tier Check (Score ≥ 90 → 10%)

Prerequisite: Create performance appraisal with score 95 for user1 (TC-PERF-01)

Actor: pm1

Steps:

1. Create new payroll for user1 after appraisal
2. Check `bonuses` field in response **Expected:**

- `bonuses = 10% of BaseSalary`

TC-PAY-11 — Bonus Tier Check (Score 75–89 → 5%)

Prerequisite: Performance appraisal score of 80 for user2

Expected:

- `bonuses = 5% of HourlyRate × hours`

TC-PAY-12 — Member Cannot Access Payroll (user1)

Actor: user1

Steps:

1. `GET /api/payrolls`

   Header: `Authorization: Bearer {user1_token}` **Expected:**

- Status `403 Forbidden`

---

12. Performance Appraisals

TC-PERF-01 — Create Appraisal for user1 (pm1)

Actor: pm1

Steps:

1. `POST /api/performance`

```
{
  "userId": "{user1_id}",
  "overallScore": 92,
  "avgTurnaroundTime": 4.5,
  "bugRate": 0.05,
  "hrComments": "Excellent performance this quarter."
}
```

Expected:

- Status `201 Created`
- `evaluatorId` set to pm1's ID
- Save returned `id` as **appraisal1_id**

TC-PERF-02 — Create Appraisal for user2 (pm1)

Actor: pm1

Steps:

1. `POST /api/performance`

```
{
  "userId": "{user2_id}",
  "overallScore": 78,
  "avgTurnaroundTime": 6,
  "bugRate": 0.12,
  "hrComments": "Good but needs improvement on bug rate."
}
```

Expected:

- Status `201 Created`

TC-PERF-03 — List Appraisals (pm1)

Actor: pm1
Steps:

1. `GET /api/performance` **Expected:**

- Status `200 OK`
- Both appraisals in results

TC-PERF-04 — Filter Appraisals by User (pm1)

Actor: pm1
Steps:

1. `GET /api/performance?userId={user1_id}` **Expected:**

- Only user1's appraisals returned

TC-PERF-05 — Get Performance Summary

Actor: user1 (any authenticated user)
Steps:

1. `GET /api/performance/summary` **Expected:**

- Status `200 OK`
- Returns average scores, turnaround times, bug rates for all employees

---

TC-PERF-06 — Member Can View Summary (user2)

Actor: user2
Steps:

1. `GET /api/performance/summary`
   Header: `Authorization: Bearer {user2_token}` **Expected:**

- Status `200 OK` (all authenticated users can access summary)

TC-PERF-07 — Member Cannot Create Appraisal (user1)

Actor: user1
Steps:

1. `POST /api/performance` (same body as TC-PERF-01)
   Header: `Authorization: Bearer {user1_token}` **Expected:**

- Status `403 Forbidden`

TC-PERF-08 — Delete Appraisal (pm2 / Admin only)

Actor: pm2
Steps:

1. `DELETE /api/performance/{appraisal1_id}` **Expected:**

- Status `200 OK` or `204`
- Appraisal no longer in list

---

13. Notifications

TC-NOTIF-01 — Get Notifications (user1)

Actor: user1 (trigger: task was assigned in TC-TASK-01)
Steps:

1. `GET /api/notifications`
   Header: `Authorization: Bearer {user1_token}` **Expected:**

- Status `200 OK`
- At least 1 notification with type `"TaskAssigned"` for task1
- Unread notifications appear first

TC-NOTIF-02 — Get Unread Count (user1)

Actor: user1
Steps:

1. `GET /api/notifications/unread-count` **Expected:**

- Status `200 OK`
- Returns integer ≥ 1

TC-NOTIF-03 — Mark Single Notification Read

Actor: user1

Steps:

1. Get a notification ID from TC-NOTIF-01 → save as **notif1_id**
2. `PATCH /api/notifications/{notif1_id}/read` **Expected:**

- Status `200 OK`
- `isRead = true` for that notification

TC-NOTIF-04 — Mark All Notifications Read

Actor: user1

Steps:

1. `PATCH /api/notifications/read-all` **Expected:**

- Status `200 OK`

2. `GET /api/notifications/unread-count` → returns `0`

TC-NOTIF-05 — Task Completion Notification (user1)

Actor: pm1 marks task Done

Steps:

1. Ensure task1 is in `InReview` (from TC-TASK-07)
2. pm1 sets task1 to `Done` (TC-TASK-08)
3. `GET /api/notifications` as user1 **Expected:**

- At least 1 notification with type `"TaskCompleted"` for task1

TC-NOTIF-06 — User Only Sees Own Notifications (user2)

Actor: user2

Steps:

1. `GET /api/notifications`
   Header: `Authorization: Bearer {user2_token}` **Expected:**

- Status `200 OK`
- Only notifications addressed to user2 (not user1's notifications)

---

14. Dashboard

TC-DASH-01 — Dashboard Summary (pm1)

Actor: pm1
Steps:

1. `GET /api/dashboard/summary`
   Header: `Authorization: Bearer {pm1_token}` **Expected:**

- Status `200 OK`
- Response includes all of these fields:
  - `activeProjects` (number)
  - `openTasks` (number)
  - `overdueTasks` (number)
  - `inReviewTasks` (number)
  - `budgetHealth` (%)
  - `efficiency` (%)
  - `taskDistribution` (by status breakdown)
  - `revenue` (total paid invoices)
  - `memberCount` (number)

TC-DASH-02 — Dashboard Summary (user1 — role-aware)

Actor: user1
Steps:

1. `GET /api/dashboard/summary`
   Header: `Authorization: Bearer {user1_token}` **Expected:**

- Status `200 OK`
- Numbers reflect only user1's scope (their own tasks/projects, not org-wide)

TC-DASH-03 — Unauthenticated Dashboard Access

Steps:

1. `GET /api/dashboard/summary` (no token) **Expected:**

- Status `401 Unauthorized`

---

15. Role-Based Access Control (RBAC) — Negative Tests

These tests verify that unauthorized users are properly blocked.

TC ID	Actor	Action	Expected
TC-RBAC-01	user1	Create project	403 Forbidden
TC-RBAC-02	user1	Delete project	403 Forbidden
TC-RBAC-03	user1	Create task	403 Forbidden
TC-RBAC-04	user1	Delete task	403 Forbidden
TC-RBAC-05	user1	Create invoice	403 Forbidden
TC-RBAC-06	user1	Update invoice status	403 Forbidden
TC-RBAC-07	user1	Access payrolls list	403 Forbidden
TC-RBAC-08	user1	Create performance appraisal	403 Forbidden
TC-RBAC-09	user1	Create profile	403 Forbidden
TC-RBAC-10	user2	Update another user's time log	403 or 404
TC-RBAC-11	user3	Change task status (not assignee)	403 Forbidden
TC-RBAC-12	user3	Send message to inaccessible channel	403 Forbidden
TC-RBAC-13	Any	Any request without JWT token	401 Unauthorized
TC-RBAC-14	Any	Any request with expired/tampered JWT	401 Unauthorized

Test Execution Checklist

Setup Order

- [ ] 1. Create pm1 & pm2 via Admin → `POST /api/profiles` with role PM
- [ ] 2. Register user1, user2, user3 (TC-AUTH-05)
- [ ] 3. Login all 5 personas and save tokens (TC-AUTH-01, 02)
- [ ] 3. Create folder1 (TC-LIST-01)
- [ ] 4. Create list1 (TC-LIST-03)
- [ ] 5. Create proj1 (TC-PROJ-01)
- [ ] 6. Add user1, user2 to proj1 (TC-PROJ-05)
- [ ] 7. Create tasks (TC-TASK-01, TC-TASK-02)

Feature Areas Coverage

Area	TCs	Priority
Authentication	TC-AUTH-01 to 11	P1 — Critical
Projects	TC-PROJ-01 to 11	P1 — Critical
Tasks	TC-TASK-01 to 12	P1 — Critical
RBAC Negative Tests	TC-RBAC-01 to 14	P1 — Critical
Notifications	TC-NOTIF-01 to 06	P2 — High
Time Logs	TC-TIME-01 to 08	P2 — High
Channels / Messages	TC-CHAN-01 to 05, TC-MSG-01 to 04	P2 — High
Task Reports	TC-REP-01 to 08	P2 — High
Payroll	TC-PAY-01 to 12	P2 — High
Invoices	TC-INV-01 to 10	P2 — High
Performance	TC-PERF-01 to 08	P3 — Medium
Files	TC-FILE-01 to 06	P3 — Medium
Dashboard	TC-DASH-01 to 03	P3 — Medium
Profiles	TC-PROF-01 to 06	P3 — Medium
Lists & Folders	TC-LIST-01 to 05	P3 — Medium

Bug Report Template

When a test fails, document it as follows:

```
TC-ID: [e.g. TC-TASK-05]
Title: [Short description]
Severity: Critical / High / Medium / Low
Actor: [e.g. user1]
Steps to Reproduce:
  1. ...
  2. ...
Expected Result: [Status code + behaviour]
Actual Result: [What actually happened]
Notes: [Response body, headers, or screenshots]
```

Last Updated: April 3, 2026